

# **Automated Document Review and Task Tracking System Using Python-Based Workflow Automation**

BITS ZG628T: Dissertation

By

**V HARSHITHA**

**(2024TM93089)**

Submitted in partial fulfilment of **M.Tech in Software Engineering**  
degree programme

Under the Supervision of

**Priyanka Nagane**

**(Accenture, Pune)**



**BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE**

**PILANI (RAJASTHAN)**

**(March, 2026)**

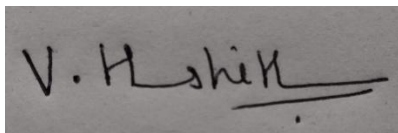
## ABSTRACT

The main goal of this project is to design and develop a secure and automated Full-stack application to manage documents and track tasks that helps to handle workflows more efficiently than in a traditional application which rely on manual monitoring. The implementation mainly focuses on automating document compliance checks, applying role-based access control to ensure data security and keeping a clear audit trail of all system activities for transparency.

In highly regulated fields such as life sciences, reviewing and approving documents is not just a routine administrative activity, it plays a major role in maintaining compliance, accuracy and overall data integrity. However, many organizations still rely on manual and error-prone processes or on rigid enterprise tools that are difficult to adapt. These traditional systems often act like black boxes, offering little transparency and limiting flexibility when workflow requirements change.

The current existing document management solutions depend on pre-configured workflow engines where customization is restricted by fixed rules and closed designs. As a result, engineers find it difficult to understand how the system works internally or to modify and improve its behavior. This highlights the need for a more open and flexible approach that clearly shows how document lifecycle management and workflow automation can be built and controlled from the ground up.

This project aims to address this challenge by developing an automated document review and task-tracking system. The solution combines a custom Python-based automation engine with a full-stack web application to create a flexible and transparent framework. Through this approach, the project demonstrates that document lifecycle management can be both efficient and easy to understand when implemented using modern software engineering practices.



---

**Signature of the Student**

**Name: V Harshitha**

**Date: 20-03-2026**

**Place: Bangalore**



---

**Signature of the Supervisor**

**Name: Priyanka Nagane**

**Date: 20-03-2026**

**Place: Pune**

## Contents

|                                                       |   |
|-------------------------------------------------------|---|
| 1. MODULES IN DOCUMENT WORKFLOW AUTOMATION .....      | 4 |
| 2. FUNCTIONAL ARCHITECTURE DIAGRAM/ DESCRIPTION ..... | 5 |
| 3. FLOW DIAGRAM .....                                 | 6 |
| 4. SEQUENCE DIAGRAM .....                             | 7 |
| 5. FEATURE SPECIFICATIONS .....                       | 7 |
| 6. DESIGN CONSIDERATIONS.....                         | 8 |
| 7. FUTURE PLAN .....                                  | 8 |
|                                                       |   |
| Figure 1: FUNCTIONAL ARCHITECTURE DIAGRAM .....       | 5 |
| Figure 2: FLOW DIAGRAM .....                          | 6 |
| Figure 3: SEQUENCE DIAGRAM.....                       | 7 |

## 1. MODULES IN DOCUMENT WORKFLOW AUTOMATION

The main modules in Document Workflow Automation are namely users, documents, tasks, audit and notifications.

| Module               | Description                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Users</b>         | It manages registration, login and role-based access control.                                             |
| <b>Documents</b>     | It manages file metadata and status of the documents (Draft, Pending, Approved).                          |
| <b>Tasks</b>         | It automatically creates review tasks and assigns them to specific users once a document is submitted.    |
| <b>Audit</b>         | It's a read-only tracker that logs every transaction, including who did what, when and on which document. |
| <b>Notifications</b> | This module sends real-time alerts to users about task assignments and approval or rejection outcomes.    |

- a) **Users Module:** The Users Module contains information about users and their roles such as ID, username, email, password and the ability to perform actions in the application based on their role as Author or Reviewer.
- b) **Documents Module:** The Documents Module is where documents will be created and stored. This includes storing the document ID, document title, document content/metadata in PDF format, who created the document (the document owner) and the current status of the document (draft, pending, approved, rejected). It will also track the life cycle of a document and maintain its status throughout the workflow.
- c) **Tasks Module:** The Tasks Module manages the task creation and assignment based on document submissions. When a document is submitted for review, a task will be automatically created and assigned to a reviewer. Review tasks include task ID, document ID, reviewer, task status and the time the task was created.
- d) **Audit Module:** The Audit Module is an audit log of all the activities performed within the application. Each log entry contains the following information: user ID, document

ID, action performed (create, update, approve, reject), timestamp. This module promotes transparency, traceability and compliance.

- e) **Notifications Module:** The Notifications Module is to provide the system with real-time alerts. Alerts include the information regarding tasks assigned to users, document status changes (approved/rejected) and other updates that may occur. Notification information will include: notification ID, user ID, message.

## 2. FUNCTIONAL ARCHITECTURE DIAGRAM/ DESCRIPTION:

The system is built with a standard full-stack architecture that is designed to be both secure and scalable.

1. **Frontend (React):** This is the user-friendly dashboard, featuring a clean and professional interface. It lets users upload documents, track their status and manage tasks easily.
2. **Backend (FastAPI):** The backend manages all the logic. It processes workflows, updates document statuses and makes sure user permissions are enforced properly.
3. **Database (SQLite/PostgreSQL):** This stores data, including user roles, documents and its states and audit history.

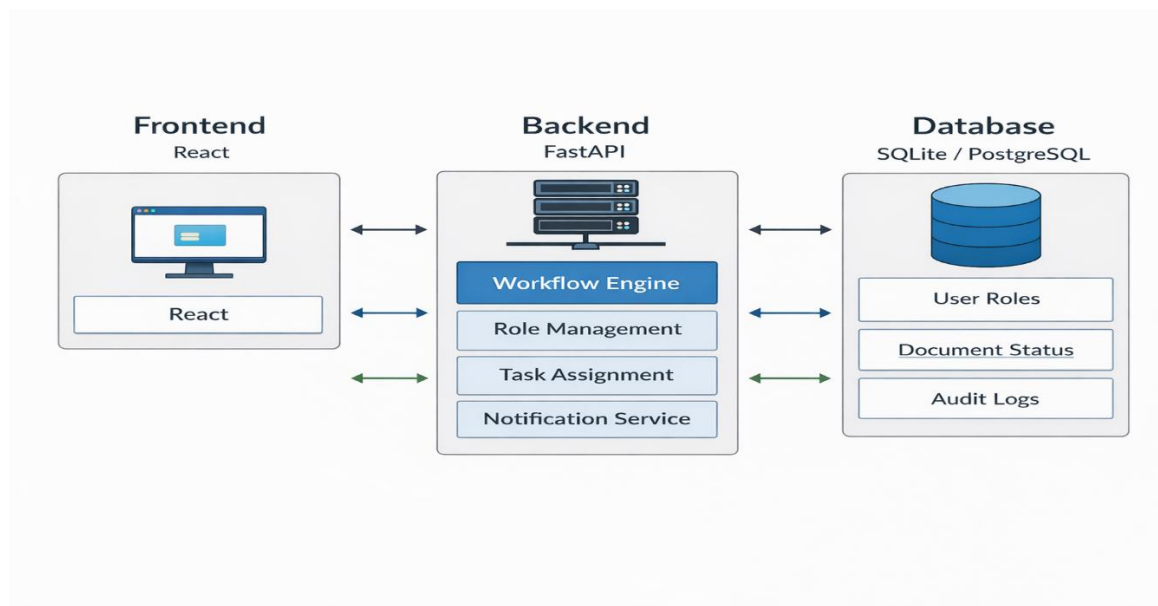


Figure 1

### 3. FLOW DIAGRAM

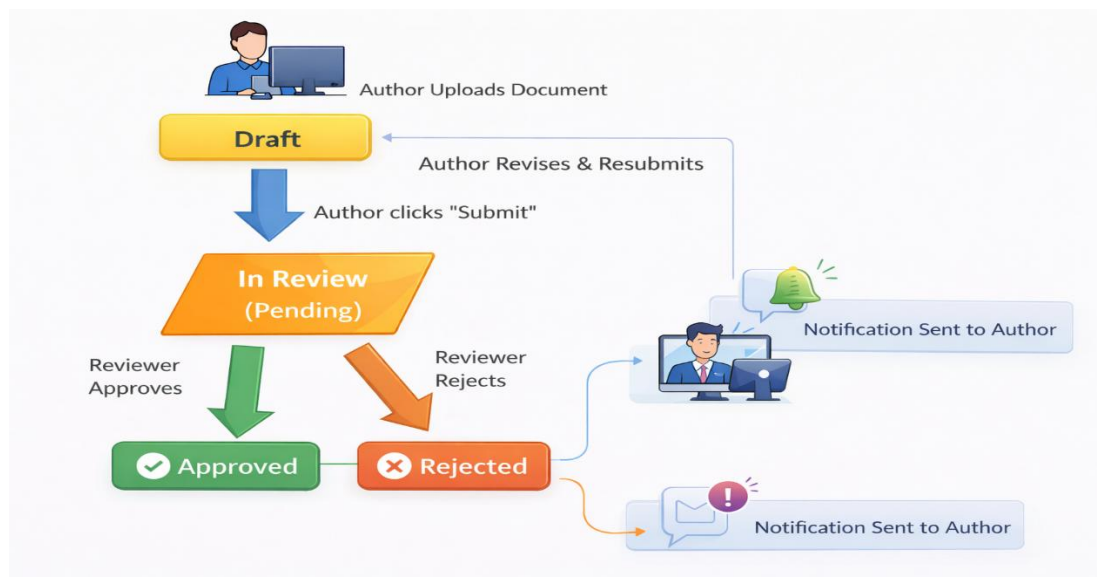


Figure 2

This diagram shows the lifecycle of a document in the system, starting from creation to approval or rejection.

The author starts the process when they upload their document which is stored in the Draft state. At this stage, the document can be edited or updated freely by the author.

The author initiates the submission process by selecting "Submit" to change the document status to In Review (Pending) state. The document now enters the state where reviewer needs to assess the document content.

- If the reviewer approves the document, it moves to the Approved state.
- If the reviewer rejects the document, it moves to the Rejected state.

After rejection, the document reverted to the Draft state which enables the author to upload and submit the document again for multiple review cycles until they achieve approval.

Also, the system notifies the author about important document updates which include approval and rejection decisions to keep them updated on the document's current status.

## 4. SEQUENCE DIAGRAM

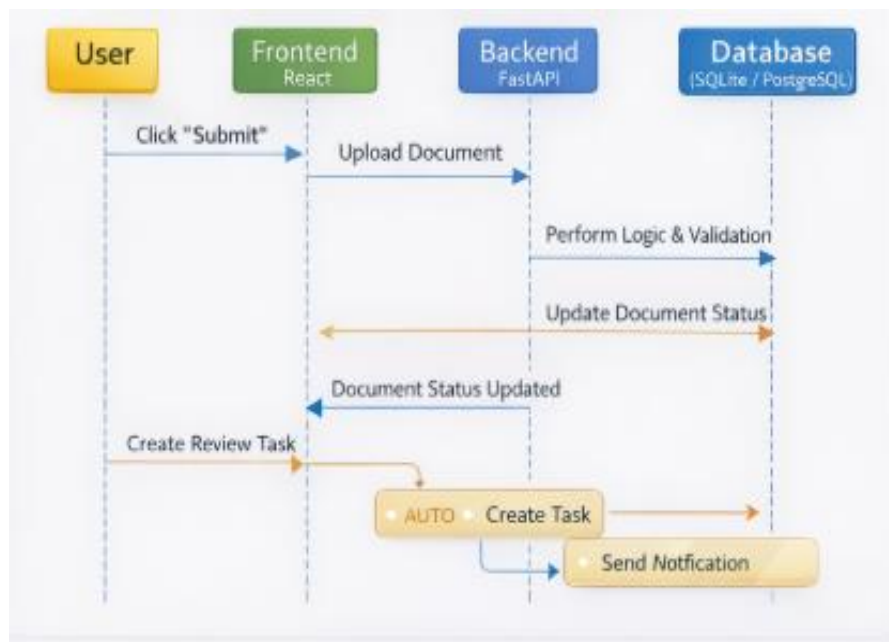


Figure 3

The diagram shows how different system components interact with each other after a document gets submitted. The user clicks "Submit" from the frontend application. The React frontend sends this request to the FastAPI backend which processes the document upload. The backend system executes its business logic together with authentication checks and document condition validation after it receives the request. The system updates the document status to Pending (In Review) after the backend completes its validation process. The system sends a response to the frontend which confirms that the document status update was successful. The system starts an automatic process that creates a review task. The system assigns this task to a reviewer who will assess the document that has been submitted. The system sends a notification to the relevant user who receives information about the new task or status update.

The document submission process begins with validation and storage followed by review assignment and user notification in an automated manner.

## 5. FEATURE SPECIFICATIONS

- **Backend Framework:** Python 3x with FastAPI
- **Database:** SQLite Production
- **Frontend Framework:** React.js with CSS
- **Workflow States:** Draft, Pending (In Review), Approved, Rejected

- **Role Permissions:** Author (Create/Edit), Reviewer (Approve/Reject)
- **Audit Trail:** Automatic timestamped logs for all POST/PATCH actions

## 6. DESIGN CONSIDERATIONS:

- **State Machine Logic:** A document cannot move to "Approved" unless a task is explicitly approved by a user with the "Reviewer" role to ensure workflow reliability,
- **Circular Workflow:** If a document is rejected, the system automatically reverts the state to "Draft" allowing the Author to revise and re-submit the document, creating a complete lifecycle loop.
- **Security:** API endpoints are protected based on user roles to prevent Authors from self-approving their own documents and the restricted content of the document are secured for confidential purposes.

## 7. FUTURE PLAN:

| Phase                       | Start - End Date | Work Done                         | Status             |
|-----------------------------|------------------|-----------------------------------|--------------------|
| <b>I: Literature Review</b> | Jan - Feb 2026   | Research on Workflow Automation   | <b>Completed</b>   |
| <b>II: Backend Dev</b>      | Feb - Mar 2026   | API, DB Schema, Workflow Engine   | <b>Completed</b>   |
| <b>III: Frontend Dev</b>    | Mar - Apr 2026   | React UI & Dashboard Integration  | <b>In Progress</b> |
| <b>IV: Testing</b>          | Apr 2026         | User Evaluation & Final Debugging | <b>Pending</b>     |
| <b>V: Submission</b>        | May 2026         | Final Dissertation Submission     | <b>Pending</b>     |

## 8. ABBREVIATIONS

- **API:** Application Programming Interface
- **RBAC:** Role-Based Access Control
- **CRUD:** Create, Read, Update, Delete